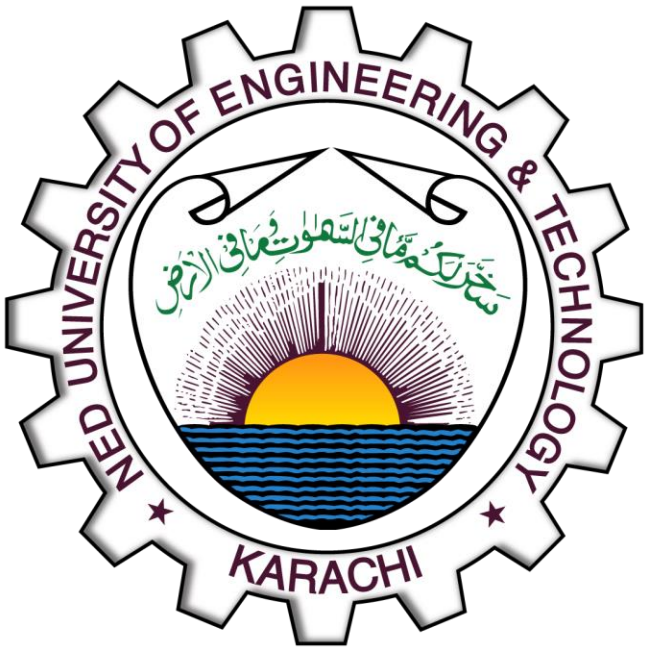
DATA STRUCTURE ALGORITHMS & APPLICATIONS

[CT-159]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: SSCS-D

LAB : #02

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

struct ListNode
{
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class List
{
    ListNode *head;

public:
    List() : head(nullptr) {}
    List(ListNode *node) : head(node) {}
    void insert(int val)
    {
        ListNode *newNode = new ListNode(val);
        if (head == nullptr)
        {
            head = newNode;
            return;
        }
        ListNode *curr = head;
        while (curr->next != nullptr)
            curr = curr->next;
        curr->next = newNode;
    }

    void print()
    {
        cout << "HEAD -> ";
        ListNode *curr = head;
        while (curr != nullptr)
        {
            cout << curr->val << " -> ";
            curr = curr->next;
        }
        cout << "NILL\n";
    }
```

```
    friend ListNode *merge(List *list1, List *list2);

    ~List()
    {
        while (head != nullptr)
        {
            ListNode *temp = head;
            head = head->next;
            delete temp;
        }
    }
};

ListNode *merge(List *list1, List *list2)
{
    // considers both list1 and List2 are sorted and takes O(m+n) space
    ListNode *curr1 = list1->head, *curr2 = list2->head;
    ListNode *dummy = new ListNode(); // dummy head to eliminate edge cases
    ListNode *curr = dummy;
    while (curr1 != nullptr || curr2 != nullptr)
    {
        ListNode *temp = nullptr;

        if (curr2 == nullptr || (curr1 != nullptr && curr1->val < curr2-
>val))
        {
            temp = new ListNode(curr1->val);
            curr1 = curr1->next;
        }
        else
        {
            temp = new ListNode(curr2->val);
            curr2 = curr2->next;
        }
        curr->next = temp;
        curr = temp;
    }
    ListNode *head = dummy->next;
    delete dummy;
    return head;
}

int main()
{
    List *list1 = new List(), *list2 = new List();
    list1->insert(1);
    list1->insert(2);
    list1->insert(4);
    cout << "List 1: ";
    list1->print();
```

```
    list2->insert(1);
    list2->insert(3);
    list2->insert(4);
    cout << "List 2: ";
    list2->print();
    cout << "MERGED:\n";
    List *merged = new List(merge(list1, list2));
    merged->print();
    return 0;
}
```

OUTPUT:

```
List 1: HEAD -> 1 -> 2 -> 4 -> NILL
List 2: HEAD -> 1 -> 3 -> 4 -> NILL
MERGED:
HEAD -> 1 -> 1 -> 2 -> 3 -> 4 -> 4 -> NILL
```

QUESTION#2:

CODE:

```
#include <iostream>
using namespace std;

struct ListNode
{
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class List
{
    ListNode *head;

public:
    List() : head(nullptr) {}
    List(ListNode *node) : head(node) {}
    void insert(int val)
    {
        ListNode *newNode = new ListNode(val);
        if (head == nullptr)
        {
            head = newNode;
            return;
        }
        ListNode *curr = head;
        while (curr->next != nullptr)
            curr = curr->next;
        curr->next = newNode;
    }

    void print()
    {
        cout << "HEAD -> ";
        ListNode *curr = head;
        while (curr != nullptr)
        {
            cout << curr->val << " -> ";
            curr = curr->next;
        }
        cout << "NILL\n";
    }
```

```
    void removeDuplicate()
    {
        // considers a sorted list.
        if (head == nullptr || head->next == nullptr)
            return;
        ListNode *curr = head->next, *prev = head;
        int currVal = head->val;
        while (curr != nullptr)
        {
            if (curr->val == currVal)
            {
                prev->next = curr->next;
                delete curr;
                curr = prev->next;
            }
            else
            {
                currVal = curr->val;
                prev = curr;
                curr = curr->next;
            }
        }
    }

    ~List()
    {
        while (head != nullptr)
        {
            ListNode *temp = head;
            head = head->next;
            delete temp;
        }
    }
};

int main()
{
    List *list = new List();
    list->insert(1);
    list->insert(1);
    list->insert(2);
    list->insert(3);
    list->insert(3);
    list->print();
    list->removeDuplicate();
    list->print();
    return 0;
}
```

OUTPUT:

```
HEAD -> 1 -> 1 -> 2 -> 3 -> 3 -> NILL
HEAD -> 1 -> 2 -> 3 -> NILL
```

QUESTION#3:

CODE:

```
#include <iostream>
using namespace std;

struct ListNode
{
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class List
{
    ListNode *head;

    ListNode *middleNode(ListNode *head)
    {
        ListNode *slow = head, *fast = head->next;
        while (fast != nullptr && fast->next != nullptr)
        {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }

    ListNode *sort(ListNode *left, ListNode *right)
    {
        ListNode dummy(0);
        ListNode *curr = &dummy;
        while (left != nullptr || right != nullptr)
        {
            if (left == nullptr || (right != nullptr && right->val < left-
>val))
            {
                curr->next = right;
                right = right->next;
            }
            else
            {
                curr->next = left;
                left = left->next;
            }
            curr = curr->next;
```

```
        }

        return dummy.next;
    }

    ListNode *divide(ListNode *head)
    {
        if (head == nullptr || head->next == nullptr)
            return head;
        ListNode *middle = middleNode(head);
        ListNode *rightHead = middle->next;
        middle->next = nullptr;
        ListNode *left = divide(head);
        ListNode *right = divide(rightHead);
        return sort(left, right);
    }

public:
    List() : head(nullptr) {}
    List(ListNode *node) : head(node) {}
    void insert(int val)
    {
        ListNode *newNode = new ListNode(val);
        if (head == nullptr)
        {
            head = newNode;
            return;
        }
        ListNode *curr = head;
        while (curr->next != nullptr)
            curr = curr->next;
        curr->next = newNode;
    }

    void mergeSort() { head = divide(head); }

    void print()
    {
        cout << "HEAD -> ";
        ListNode *curr = head;
        while (curr != nullptr)
        {
            cout << curr->val << " -> ";
            curr = curr->next;
        }
        cout << "NILL\n";
    }
};

int main()
```

```
{
    List *list = new List();
    list->insert(4);
    list->insert(2);
    list->insert(1);
    list->insert(3);
    cout << "UNSORTED:\n";
    list->print();
    list->mergeSort();
    cout << "SORTED:\n";
    list->print();
    return 0;
}
```

OUTPUT:

```
UNSORTED:
HEAD -> 4 -> 2 -> 1 -> 3 -> NILL
SORTED:
HEAD -> 1 -> 2 -> 3 -> 4 -> NILL
```

QUESTION#4:

CODE:

```
#include <iostream>
#include <stack>
using namespace std;

struct ListNode
{
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class List
{
    ListNode *head;

public:
    List() : head(nullptr) {}
    List(ListNode *node) : head(node) {}
    void insert(int val)
    {
        ListNode *newNode = new ListNode(val);
        if (head == nullptr)
        {
            head = newNode;
            return;
        }
        ListNode *curr = head;
        while (curr->next != nullptr)
            curr = curr->next;
        curr->next = newNode;
    }

    ListNode *middleNode()
    {
        ListNode *slow = head, *fast = head->next, *prev = nullptr;
        while (fast != nullptr && fast->next != nullptr)
        {
            prev = slow;
            slow = slow->next;
            fast = fast->next->next;
        }
        if (fast == nullptr)
        {
```

```
            prev->next = nullptr;
            return slow->next;
        }
        else
        {
            prev = slow->next;
            slow->next = nullptr;
            return prev;
        }
    }

    bool palindrome()
    {
        if (head == nullptr) return true;
        stack<int> check;
        ListNode *right = middleNode();
        while (head != nullptr)
        {
            check.push(head->val);
            head = head->next;
        }
        while (right != nullptr)
        {
            if (check.empty() || check.top() == right->val)
                check.pop();
            else
                return false;
            right = right->next;
        }
        return true;
    }

    void print()
    {
        cout << "HEAD -> ";
        ListNode *curr = head;
        while (curr != nullptr)
        {
            cout << curr->val << " -> ";
            curr = curr->next;
        }
        cout << "NILL\n";
    }
};

int main()
{
    List *list = new List();
    list->insert(1);
    list->insert(2);
```

```
    list->insert(2);
    list->insert(1);
    cout << "LIST:\n";
    list->print();
    cout << "IS PALINDROME: " << (list->palindrome() ? "yes" : "no");
    cout << endl;
    return 0;
}
```

OUTPUT:

```
LIST:
HEAD -> 1 -> 2 -> 2 -> 1 -> NILL
IS PALINDROME: yes
```

QUESTION#5:

CODE:

```
#include <iostream>
using namespace std;

struct ListNode
{
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class CircularQueue
{
    ListNode *rear;

public:
    CircularQueue() : rear(nullptr) {}

    bool isEmpty() { return rear == nullptr; }

    void enqueue(int val)
    {
        ListNode *newNode = new ListNode(val);
        cout << "ENQUEUED..." << newNode->val << endl;
        if (isEmpty())
        {
            newNode->next = newNode;
            rear = newNode;
            return;
        }
        newNode->next = rear->next;
        rear->next = newNode;
        rear = newNode;
    }

    void dequeue()
    {
        if (isEmpty())
        {
            cout << "Queue Underflow...\n";
            return;
        }
        ListNode *front = rear->next;
        cout << "DEQUEUED..." << front->val << endl;
```

```
        if (rear == front)
            rear = nullptr;
        else
            rear->next = front->next;

        delete front;
    }

    int peek()
    {
        if (isEmpty())
        {
            cout << "Queue is empty!\n";
            return -1;
        }
        return rear->next->val;
    }

    void print()
    {
        if (isEmpty())
        {
            cout << "Queue is empty...\n";
            return;
        }
        cout << "FRONT -> ";
        ListNode *curr = rear->next;
        do
        {
            cout << curr->val << " -> ";
            curr = curr->next;
        } while (curr != rear->next);
        cout << "REAR\n";
    }

    ~CircularQueue()
    {
        while (!isEmpty())
            dequeue();
    }
};

int main()
{
    CircularQueue *queue = new CircularQueue();
    queue->print();
    queue->enqueue(1);
    queue->enqueue(2);
    queue->enqueue(3);
    queue->enqueue(4);
```

```
    queue->enqueue(5);
    queue->print();
    queue->dequeue();
    queue->dequeue();
    queue->dequeue();
    queue->print();
    cout << "The element at the front is: " << queue->peek() << endl;
    return 0;
}
```

OUTPUT:

```
Queue is empty...
ENQUEUED...1
ENQUEUED...2
ENQUEUED...3
ENQUEUED...4
ENQUEUED...5
FRONT -> 1 -> 2 -> 3 -> 4 -> 5 -> REAR
DEQUEUED...1
DEQUEUED...2
DEQUEUED...3
FRONT -> 4 -> 5 -> REAR
The element at the front is: 4
```

QUESTION#6:

CODE:

```
#include <iostream>
using namespace std;

struct ListNode
{
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class Stack
{
private:
    ListNode *head;

public:
    Stack() : head(nullptr) {}
    Stack(ListNode *node) : head(node) {}

    int peak() { return head->val; }
    bool isEmpty() { return head == nullptr ? true : false; }

    void push(int val)
    {
        ListNode *newNode = new ListNode(val);
        if (head == nullptr)
        {
            head = newNode;
            return;
        }
        newNode->next = head;
        head = newNode;
    }
    void pop()
    {
        ListNode *temp = head;
        head = head->next;
        delete temp;
    }

    void print()
    {
        if (isEmpty())
```

```
        {
            cout << "EMPTY STACK.\n";
            return;
        }
        cout << "TOP -> ";
        ListNode *curr = head;
        while (curr != nullptr)
        {
            cout << curr->val << ", ";
            curr = curr->next;
        }
        cout << "END\n";
    }

    ~Stack()
    {
        while (!isEmpty())
            pop();
    }
};

int main()
{
    Stack *stack = new Stack();
    stack->print();
    for (int i = 10; i < 51; i += 10)
        stack->push(i);
    stack->print();
    for (int i = 0; i < 5; i++)
    {
        cout << stack->peak() << endl;
        stack->pop();
        stack->print();
    }
    return 0;
}
```

OUTPUT:

```
EMPTY STACK.
TOP -> 50, 40, 30, 20, 10, END
50
TOP -> 40, 30, 20, 10, END
40
TOP -> 30, 20, 10, END
30
TOP -> 20, 10, END
20
TOP -> 10, END
10 EMPTY STACK.
```